

components for respective functions.

Now, let's head towards the blocks editor to define the various kinds of behaviour. We need to discuss the actual functionality that we are expecting from our application.

1. First, the user will be writing a text into the text box.
2. On clicking the 'generate code' button, it should call the Web API to return the generated code.
3. On clicking the 'Share code' button, a list should pop up with all the sharing applications being available on the device like Bluetooth, WhatsApp, Shareit, Xender, etc.
4. The *Clear* code button should clear the canvas image.
5. The *Save to SD card* button should save the image to the SD card.

So let's move on and add these behaviours, using the block editor. I hope you remember how to switch from the designer to the block editor. There is a button available right above the *Properties* pane to help you do this.

Block editor blocks: I have already prepared the blocks for you. All you need to do is drag the relevant blocks from the left side palette and drop them on the viewer. Arrange the blocks in the same way as shown in Figure 4. I will explain each one, telling you what it does and how it is called.

On clicking the 'Generate code' button, we are calling the Web API located at <https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=>. By using the REST procedure of the Web component, we will receive the image from the URL source and the same will be set as an image on the canvas.

Let's save the image to the SD card with a common name. If the file name already exists in the SD card, it will overwrite the previous file to create a new one. Using the sharing component, we will pick the file stored in the SD card.



Figure 4: Block editor image 1



Figure 5: Block editor image 2



Figure 6: Block editor image 3

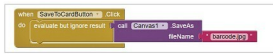


Figure 7: Block editor image 4

On clicking the 'Clear code' button, the canvas image should be cleared.

On selecting 'Save to SD card', the image should be saved to the SD card with the name *barcode.jpg*.

Packaging and testing: To test the app, you need to get it on your phone. First, you have to download the application to your computer and then move it to your phone via Bluetooth or USB cable. I'll tell you how to download it.

1. On the top row, click on the *Build* button. It will give you an option to download the APK to your computer.
2. You will be able to see the progress of the download, and after it has been successfully completed, the application will be placed in the download folder of your directory or the location you have set for it.
3. Now you need to get this APK file to your mobile phone either via Bluetooth or via USB cable. Once you have placed the APK file in your

SD card, you need to install it.

Follow the on-screen instructions to install it. You might get some notification or warning saying, 'Install from un-trusted source'.

Allow this from the settings and after successful installation, you will see the icon of your application in the menu of your mobile. Here, you will see the default icon, which can be changed and we will tell you how to do that as we move ahead in this course.

I hope your application is working exactly as per your requirements. Now, depending upon your usability and customisation, you can change various things like the image, sound and behaviour, as well.

Debugging the application:

We have just created the prototype of the application with very basic functionality but what else might the user be interested in? Now come various use cases, which require serious attention so as not to annoy the user, and your app should be able to address these. Consider the following cases:

1. Can we think of a way to store files with unique names?
2. If we are saving multiple codes to the SD card, can we ask the user to browse a particular image and then share it?
3. Can we extend the same application to read the barcode as well?

These are some scenarios that might occur and users will be pretty happy seeing these implemented.

Think about these scenarios, and how you can integrate them into the application. Do ask me if you fail to address any of the above cases.

You have successfully built another useful Android app for yourself.

Happy inventing! 

By: Meghraj Singh Beniwal

The author has a B. Tech in electronics and communication, is a freelance writer and an Android app developer. He is currently working as an automation engineer at Infosys, Pune. He can be contacted at meghraj Singh01@rediffmail.com or meghrajwithandroid@gmail.com.